Escrevendo no Quadro Negro

time limit per test: 2 segundos
memory limit per test: 256 megabytes
input: entrada padrão
output: saída padrão

Te é dado uma sequência de N números:

$$A = [A_1, A_2, \ldots, A_N].$$

Aqui cada, A_i $(1 \leq i \leq N)$ satisfaz $1 \leq A_i \leq N$.

Aoki e Takahashi vão jogar N rodadas de um jogo. Para cada $i = 1, 2, \ldots, N$, o i-ésimo jogo será jogado como segue.

1. Aoki especifica um inteiro positivo K_i.
2. Depois de saber K_i que Aoki especificou, Takahashi escolhe um inteiro S_i entre 1 e N, inclusive, e escreve no quadro negro.
3. Repita o seguinte K_i vezes.
    - Troque o inteiro x escrito no quadro negro com A_x.

Se i está escrito no quadro negro depois de K_i iterações, Takahashi ganha a i-ésima rodada; senão, Aoki ganha. Aqui, K_i e S_i podem ser escolhidos independentemente para cada $i = 1, 2, \ldots, N$.

Ache o número de rodadas que Takahashi ganha se ambos os jogadores jogarem otimamente para ganhar.

Input

A primeira linha da entrada contém o inteiro N $(1 \leq N \leq 2 \cdot 10^5)$.

A segunda linha contém os N inteiros A_i $(1 \leq i, A_i \leq N)$.

Output

Imprima o número de rodadas que Takahashi ganha se ambos os jogadores jogarem otimamente para ganhar.

Examples

input
3 2 2 3
output
2

input
2 2 1
output

2

Note

No primeiro exemplo, na primeira rodada, se Aoki especifica que $K_1 = 2$, Takahashi não pode vencer em nenhum número que ele escolha pra S_1: 1, 2 ou 3.

Por exemplo, se Takahashi escreve $S_1 = 1$ no quadro negro inicial, as duas operações irão mudar este número como segue: $1 \to 2$ $(= A_1)$, $2 \to 2$ $(= A_2)$. O número final no quadro negro será 2 $(\neq 1)$, então Aoki ganha.

Porém, na segunda e a na terceira rodada, Takahashi pode ganhar escrevendo 2 e 3, respectivamente, no quadro negro inicial, independente do valor que Aoki especificar como K_i.

Assim, se ambos os jogadores jogarem otimamente para ganhar, Takashi ganha duas rodadas: a segunda e a terceira. Então a resposta a ser impressa é 2.

No segundo exemplo, Takahashi pode ganhar na primeira rodada escrevendo 2 no quadro negro inicial se K_1 especificado por Aoki for ímpar, e 1 se for par. Similarmente, existe uma maneira de Takahashi a segunda rodada. Assim, Takahashi é capaz de ganhar as duas rodadas: a resposta é 2.

Descobrindo Artefatos

time limit per test: 1 segundo
memory limit per test: 256 megabytes
input: entrada padrão
output: saída padrão

Johnny está jogando um conhecido jogo de computador. O jogo está em algum país, onde o jogador pode viajar livremente, fazer missões e ganhar experiência.

Nesse país existem N ilhas e M pontes entre elas, então você pode viajar de uma ilha para outra. No meio de algumas pontes encontram-se artefatos antigos e poderosos. Johnny não está interessado nos artefatos, mas pode ganhar algum dinheiro vendendo eles.

No início Johnny está na ilha A e o negociante de artefatos está na ilha B (possivelmente eles estão na mesma ilha). Johnny quer encontrar algum artefato, ir ao negociante e vendê-lo. A única dificuldade é que as pontes são muito velhas e se destroem logo após passar por elas. O personagem de Johnny não sabe nadar, voar ou se teletransportar, então o problema se tornou muito difícil.

Observe que Johnny não pode passar pela metade da ponte, coletar o artefato e voltar para a mesma ilha.

Determine se Johnny pode encontrar algum artefato e vendê-lo.

Input

A primeira linha contém dois inteiros N e M $(1 \leq n \leq 3 \cdot 10^5, 0 \leq M \leq 3 \cdot 10^5)$ — o número de ilhas e pontes no jogo.

Cada uma das próximas M linhas contém a descrição da ponte — três inteiros X_i, Y_i, Z_i $(1 \leq X_i, Y_i \leq N, X_i \neq Y_i, 0 \leq Z_i \leq 1)$, onde X_i e Y_i são as ilhas conectadas pelo i-ésima ponte, Z_i é igual a 1 se essa ponte contiver um artefato e a 0 caso contrário. Não há mais de uma ponte entre qualquer par de ilhas. É garantido que é possível viajar entre qualquer par de ilhas.

A última linha contém dois inteiros A e B $(1 \leq A, B \leq N)$ — as ilhas onde estão Johnny e o negociante de artefatos, respectivamente.

Output

Se Johnny pode achar algum artefato e vendê-lo imprima apenas a palavra "`YES`" (sem aspas). Senão, imprima a palavra "`NO`" (sem aspas).

Examples

input
6 7 1 2 0 2 3 0 3 1 0 3 4 1 4 5 0 5 6 0

```
6 4 0
1 6
```

output

```
YES
```

input

```
5 4
1 2 0
2 3 0
3 4 0
2 5 1
1 4
```

output

```
NO
```

input

```
5 6
1 2 0
2 3 0
3 1 0
3 4 0
4 5 1
5 3 0
1 2
```

output

```
YES
```

Chuva versus ETDC

time limit per test: 1 segundo
memory limit per test: 256 megabytes
input: entrada padrão
output: saída padrão

A ETDC (Empresa de Telefonia Desnecessariamente Cara) foi contratada para interligar as diversas universidades da Nlogônia. As conexões são bidirecionais e permitem que duas universidades possam acessar dados guardados em qualquer um dos lados. O problema é que como as universidades estão longe uma das outras, a fiação é muito longa e muito custosa, e em muito casos, a empresa decidiu deixar algumas universidades dependendo de só um fio para se conectar com as outras.

Mas devido ao regime intenso de chuvas, a ETDC está preocupada que a pesquisa pode ser interrompida devido a sua falta de planejamento. Membros da ETDC já recorreram a meios alternativos para fazer parar de chover, mas a parte técnica da agência não acredita que isso vai funcionar, mesmo sendo prática recorrente do governo da Nlogônia. Ajude a ETDC a achar os cabos que se desconectados, impediriam pelo menos uma universidade de se comunicar com as outras universidades.

Input

A entrada começa com uma linha contendo um inteiro T $(1 \leq T \leq 1000)$, o número de casos de teste. Cada caso de teste começa com os números N $(2 \leq N \leq 100)$ e M $(1 \leq M \leq \frac{N \times (N-1)}{2})$, o número de universidade e o número de cabos, respectivamente; seguem M linhas com dois inteiros X e Y $(1 \leq X, Y \leq N)$ indicando que existe uma conexão entre as universidades X e Y.

Output

Para cada caso de teste, imprima um único número inteiro, o número de cabos que causariam bloqueio na comunicação entre as universidades.

Examples

input
4 3 2 1 2 3 2 3 2 2 1 2 3 3 3 2 1 1 3 3 2 3 3 2 1 2 3 3 1
output

```
2
2
0
0
```

input

```
2
5 4
5 1
5 2
5 3
5 4
6 5
2 1
2 3
5 4
5 6
5 2
```

output

```
4
5
```

Bloqueando a Desinformação

time limit per test: 1 segundo
memory limit per test: 256 megabytes
input: entrada padrão
output: saída padrão

As redes sociais permitiram que todo tipo de informação se espalhasse muito rapidamente e também permitiu que qualquer pessoa tenha voz. O poder da difusão massivo que era reservado apenas ao jornais e a televisão agora está disponível para qualquer usuário da rede. As notícias agora se espalhem nos mensageiros, através de grupos de colegas, amigos ou grupos onde pessoas compartilham interesses mútuos. Porém, diferente dos jornais que tomam a responsabilidade pelo que foi impresso, verificar a fonte de uma determinada informação nessa rede boca a boca é bem mais difícil.

Kage conseguiu obter através da análise dos metadados que o seu mensageiro coleta, o padrão de encaminhamento das mensagens de pessoas. Neste padrão, é possível identificar fontes de informação: Pessoas que não recebem mensagens encaminhadas. Kage ficou interessado em saber quantas pessoas não conseguiriam identificar com exatidão a fonte de suas informações (isto é, existe mais de uma fonte de informação possível para esta pessoa) e pediu sua ajuda.

Input

A primeira linha contém dois inteiros sendo eles N $(0 \leq N \leq 10^5)$, o número de pessoas da rede, M $(0 \leq M \leq 2 \cdot 10^5)$, o número de pares distintos de pessoas que encaminharam mensagens.

Seguem M linhas, cada uma com dois inteiros U e V $(1 \leq U, V \leq N, U \neq V)$ que indica que a pessoa U encaminha mensagens para a pessoa V. Não há pares repetidos.

Output

Você deve imprimir a quantidade de pessoas que não conseguiriam saber exatamente daonde as suas informações vem.

Examples

input
9 8 1 2 2 4 3 4 4 9 5 6 6 7 7 8 8 6
output
2

input
9 6 1 2 2 3

```
5 6
6 7
7 8
7 9
```

output

```
0
```

input

```
3 3
1 2
1 3
2 3
```

output

```
0
```

input

```
6 7
1 2
2 3
3 1
4 5
5 6
6 4
1 6
```

output

```
0
```

Ajudando o Trânsito

time limit per test: 1 segundo
memory limit per test: 256 megabytes
input: entrada padrão
output: saída padrão

O presidente da Nlogônia decidiu, por decreto, que todas as ruas da Nlogônia deveriam ser de mão única. Devido a certo desconhecimento de ciência básica, não houve um planejamento adequado para as mudanças. Depois que o novo sistema entrou em vigor, as pessoas não eram capazes de ir trabalhar, ou não conseguiam voltar para casa do trabalho, por exemplo. Como resultado, houve caos e rebeliões em muitas cidades.

O presidente sofreu impeachment e a nova administração do país contratou uma equipe de cientistas para resolver o problema. Por sua vez, essa equipe contratou você, especialista em complexidade de algoritmos, para ajudá-los no cálculo eficiente de soluções.

Assim, para cada cidade, você recebe os pontos de referência da cidade e as ruas de mão única, cada uma conectando dois pontos de referência. Sua tarefa é determinar o número mínimo de pontes de mão única que devem ser construídas para ter conectividade total na cidade. Cada ponte também deve conectar dois pontos de referência.

Input

A primeira linha da entrada contém dois inteiros, N e M $(1 \leq N \leq 10^4, 0 \leq M \leq 10^6)$, onde N é o número de pontos de referência e M é o número de ruas. Cada uma das próximas M linhas contém dois inteiros, R e S, $1 \leq R, S \leq N, R \neq S$, que corresponde a uma rua que liga R a S, tal que todos os veículos daquela rua se deslocam de R em direção a S.

Output

Seu programa deve imprimir uma única linha contendo o número mínimo de pontes necessárias para deixar os habitantes felizes.

Examples

input
7 7 1 2 2 3 3 1 6 1 6 4 4 5 7 6
output
2

input
7 7 2 1 3 2 1 3

```
1 6
4 6
5 4
6 7
```

output

```
2
```

input

```
2 1
1 2
```

output

```
1
```

input

```
3 3
1 2
2 3
3 1
```

output

```
0
```

input

```
2 0
```

output

```
2
```

input

```
6 4
1 2
1 3
4 6
5 6
```

output

```
3
```